

LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 10

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

Bab 1. Pendahuluan

1.1 Latar Belakang

Object-Oriented Programming (OOP) merupakan pendekatan pemrograman yang menggunakan objek dan class sebagai dasar dalam membangun sebuah aplikasi. Dengan OOP, program dapat disusun menjadi beberapa bagian yang memiliki tanggung jawab masing-masing sehingga kode menjadi lebih terstruktur, mudah dipahami, dan mudah dikembangkan.

Salah satu konsep penting dalam OOP adalah **Inheritance** atau **pewarisan**. Inheritance memungkinkan sebuah class mewarisi atribut dan method dari class lain. Dengan konsep tersebut, kode yang memiliki fungsi yang sama tidak perlu ditulis berulang kali pada setiap class. Hal ini dapat mengurangi duplikasi kode dan mempermudah proses pemeliharaan aplikasi.

Dalam aplikasi berbasis MVC, beberapa Controller biasanya memiliki fungsi yang sama, seperti menampilkan halaman menggunakan method `view()` dan mengarahkan pengguna menggunakan method `redirect()`. Apabila setiap Controller membuat method tersebut secara mandiri, akan terjadi pengulangan kode. Oleh karena itu, dibuat sebuah `BaseController` sebagai parent class yang menyediakan method umum untuk Controller lainnya.

Selain Inheritance, praktikum ini juga menerapkan **Repository Pattern**. Repository Pattern merupakan pola desain yang digunakan untuk memisahkan proses akses database dari Controller. Seluruh perintah SQL yang berkaitan dengan data mahasiswa ditempatkan di dalam `MahasiswaRepository`. Controller hanya memanggil method Repository seperti `all()`, `find()`, `create()`, `update()`, dan `delete()` tanpa mengetahui detail query SQL yang digunakan.

Dengan menerapkan kedua konsep tersebut, aplikasi Manajemen Data Mahasiswa memiliki struktur kode yang lebih terorganisasi. Controller berfokus pada alur aplikasi, sedangkan Repository bertanggung jawab terhadap akses data dan `BaseController` menyediakan fungsi umum yang dapat diwariskan oleh Controller lainnya.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk:

1. Memahami konsep **Inheritance** dalam pemrograman berorientasi objek.
2. Membuat `BaseController` yang menyediakan method umum `view()` dan `redirect()`.
3. Menerapkan inheritance pada `MahasiswaController`.

4. Memahami konsep **Repository Pattern**.
5. Membuat MahasiswaRepository untuk menangani akses data mahasiswa.
6. Memisahkan query SQL dari Controller.
7. Menerapkan Dependency Injection antara Database, Repository, dan Controller.
8. Menguji fitur tampil, tambah, ubah, dan hapus data mahasiswa.
9. Menghasilkan aplikasi yang lebih rapi, mudah dipelihara, dan mudah dikembangkan.

Bab 2. Dasar Teori

2.1 Inheritance

Inheritance atau pewarisan adalah konsep OOP yang memungkinkan sebuah class mewarisi atribut dan method dari class lainnya. Pada aplikasi MVC, fungsi yang sama pada beberapa Controller dapat ditempatkan pada sebuah parent class, yaitu BaseController.

Penerapan inheritance memberikan beberapa keuntungan, yaitu:

- Mengurangi duplikasi kode.
- Menerapkan prinsip **DRY (Don't Repeat Yourself)**.
- Mempermudah pemeliharaan aplikasi.
- Method umum cukup dibuat satu kali pada parent class.
- Perubahan pada fungsi umum dapat dilakukan pada parent class.
- Controller menjadi lebih sederhana.

Pada praktikum ini struktur inheritance yang digunakan adalah:

BaseController -> Controller -> Mahasiswa Controller

2.2 Repository Pattern

Repository Pattern merupakan pola desain yang digunakan untuk memisahkan logika akses database dari bagian aplikasi lainnya.

Pada aplikasi ini, seluruh query SQL mahasiswa ditempatkan pada: MahasiswaRepository.php
Repository menyediakan beberapa method seperti:

all(), find(), create(), update(), delete()

Controller tidak menuliskan query SQL secara langsung.

Contohnya:

```
$mahasiswa =
```

```
    $this->repo->all();
```

atau:

```
$this->repo->create(
```

```
    $mahasiswa
```

```
);
```

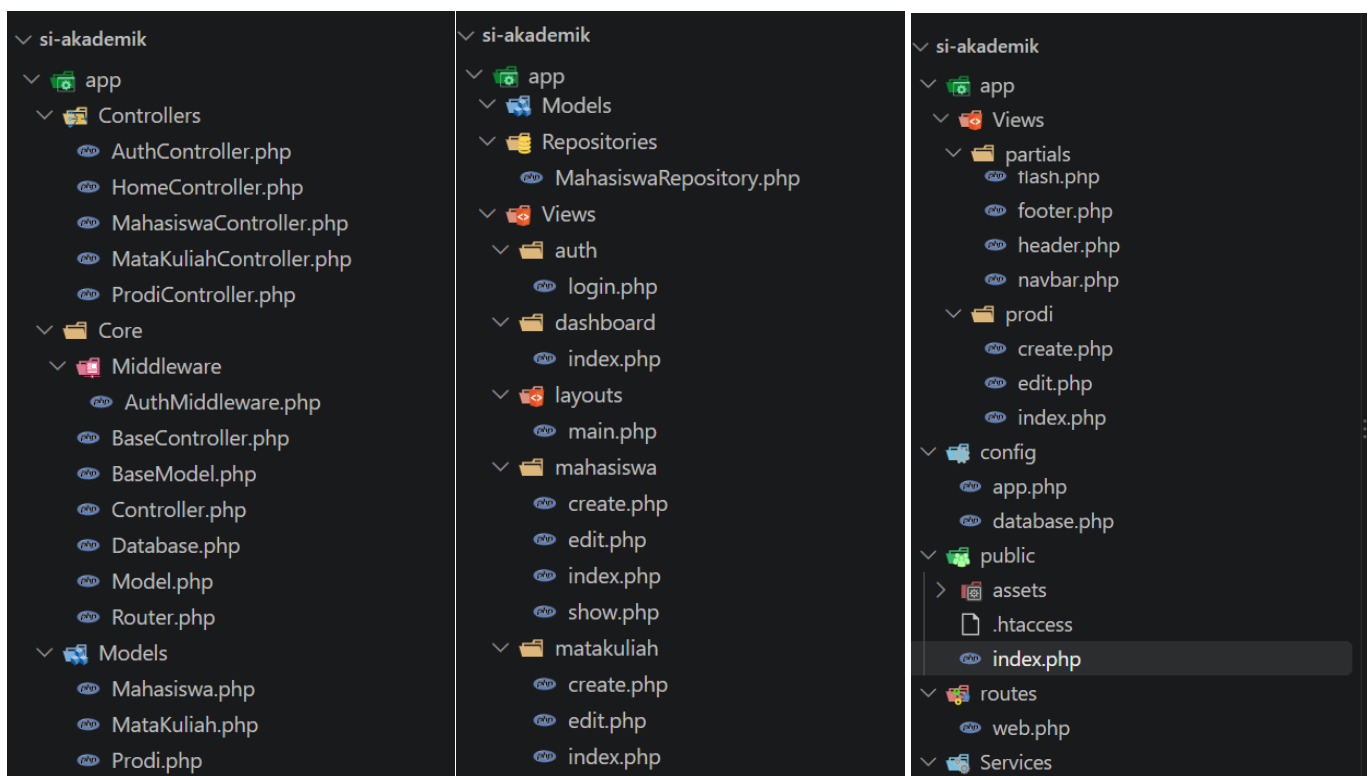
Dengan demikian, Controller hanya berinteraksi dengan method Repository tanpa mengetahui bagaimana query database dijalankan.

Penerapan Repository Pattern memberikan beberapa keuntungan, yaitu:

- Memisahkan logika aplikasi dengan logika akses database.
- Membuat Controller menjadi lebih sederhana.
- Mengurangi ketergantungan Controller terhadap SQL.
- Mempermudah pemeliharaan kode.
- Mempermudah pengujian program.
- Mempermudah perubahan struktur database atau mekanisme penyimpanan data.

Bab 3. Hasil Praktikum

3.1 Struktur Folder



3.2 Implementasi Base Controller

```
app > Core > BaseController.php > {} App\Core > BaseController
1  <?php
2
3  namespace App\Core;
4
5  1 reference | 6 implementations
6  abstract class BaseController
7  {
8      /**
9       * Menentukan base path aplikasi.
10      */
11      1 reference | 0 overrides
12      protected function basePath(): string
13      {
14          return rtrim(
15              str_replace(
16                  '\\',
17                  '/',
18                  dirname(
19                      $_SERVER['SCRIPT_NAME'] ?? '/index.php'
20                  )
21              ),
22              '/';
23      }
```

```
27  4 references | 0 overrides
28  protected function view(
29      string $view,
30      array $data = []
31  ): void {
32      extract($data);
33
34      require __DIR__ .
35          '/../Views/' .
36          $view .
37          '.php';
38
39      /**
40       * Redirect menggunakan nama method baru.
41       */
42      7 references | 0 overrides
43      protected function redirect(
44          string $url
45      ): void {
46          header(
47              'Location: ' .
48              $this->basePath() .
49              $url
50          );
51      }
```

Method utama yang digunakan adalah view() dan redirect(). Method redirectTo() digunakan untuk menjaga kompatibilitas dengan Controller lama yang masih menggunakan nama method tersebut

3.3 Implementasi Controller

```
app > Core > Controller.php > {} App\Core > Controller
1  <?php
2
3  namespace App\Core;
4
5  10 references | 5 implementations
   abstract class Controller extends BaseController
6  {
7      //
8  }
```

Class Controller mewarisi BaseController.

3.4 Implementasi Mahasiswa Controller

```
2 references | 0 implementations
11  class MahasiswaController extends Controller
12  {
13      /**
14       * Repository yang digunakan Controller.
15       */
16      12 references
       private MahasiswaRepository $repo;
17
18
19      /**
20       * Dependency Injection.
21       */
22      1 reference | 0 overrides
       public function __construct(
23         MahasiswaRepository $repo
24     ) {
25         $this->repo = $repo;
26     }
27 }
```

Repository diberikan kepada Controller melalui constructor.

a. Menampilkan Data

```
32 ✓ public function index(): void
33 {
34     $keyword = trim(
35         $_GET['q'] ?? ''
36     );
37
38
39     $page = max(
40         1,
41         (int) (
42             $_GET['page'] ?? 1
43         )
44     );
45
46
47     $perPage = 5;
48
49
50     $total = $this->repo->count(
51         $keyword !== ''
52         ? $keyword
53         : null
54     );
```

Pada kode tersebut Controller tidak menjalankan SQL.

Controller hanya memanggil: `$this->repo->all()`;

Kemudian menggunakan method inheritance: `$this->view()`;

3.5 Implementasi Mahasiswa Repository

```
5 references | 0 implementations
9 ✓ class MahasiswaRepository
10 {
11     8 references
12     private PDO $db;
13
14     /**
15      * Dependency Injection.
16      *
17      * Database diberikan dari luar.
18      */
19     1 reference | 0 overrides
20     public function __construct(
21         Database $database
22     ) {
23         $this->db =
24             $database->getConnection();
25     }
```

Query SQL berada di Repository, bukan di Controller

a. Method Menampilkan Data

```
10 public function all(  
11     ?string $keyword = null,  
12     int $limit = 5,  
13     int $offset = 0  
14 ): array {  
15  
16     $sql = "  
17         SELECT  
18             m.id,  
19             m.nim,  
20             m.nama,  
21             m.email,  
22             m.prodi_id,  
23             m.angkatan,  
24             m.status,  
25             m.updated_at,  
26  
27             p.kode AS prodi_kode,  
28             p.nama AS prodi_nama  
29  
30         FROM mahasiswa m  
31  
32         INNER JOIN prodi p  
33             ON p.id = m.prodi_id
```

b. Method Find

```
4 references | 0 overrides  
174 public function find(  
175     int $id  
176 ): ?array {  
177  
178     $stmt = $this->db->prepare("  
179         SELECT  
180             m.id,  
181             m.nim,  
182             m.nama,  
183             m.email,  
184             m.prodi_id,  
185             m.angkatan,  
186             m.status,  
187             m.updated_at,  
188  
189             p.kode AS prodi_kode,  
190             p.nama AS prodi_nama  
191  
192         FROM mahasiswa m  
193  
194         INNER JOIN prodi p
```

c. Method Create

```
237 public function create(  
238     Mahasiswa $mahasiswa  
239 ): bool {  
240  
241     $stmt = $this->db->prepare("  
242         INSERT INTO mahasiswa  
243         (  
244             nim,  
245             nama,  
246             email,  
247             prodi_id,  
248             angkatan,  
249             status  
250         )  
251         VALUES  
252         (  
253             :nim,  
254             :nama,  
255             :email,  
256             :prodi_id,  
257             :angkatan,  
258             :status  
259         )  
260     ");
```

d. Method Update

```
288 public function update(  
289     int $id,  
290     Mahasiswa $mahasiswa  
291 ): bool {  
292  
293     $stmt = $this->db->prepare("  
294         UPDATE mahasiswa  
295         SET  
296             nim = :nim,  
297             nama = :nama,  
298             email = :email,  
299             prodi_id = :prodi_id,  
300             angkatan = :angkatan,  
301             status = :status  
302  
303         WHERE id = :id  
304     ");  
305
```

e. Method Delete

```
335 public function delete(  
336     int $id  
337 ): bool {  
338  
339     $stmt = $this->db->prepare("  
340         DELETE FROM mahasiswa  
341         WHERE id = :id  
342     ");  
343  
344  
345     return $stmt->execute([  
346         'id' => $id  
347     ]);  
348 }  
349 }
```

3.6 Implementasi Dependency Injection

Dependency Injection digunakan agar Controller tidak membuat Repository sendiri.

Pada MahasiswaController:

```
22 public function __construct(  
23     MahasiswaRepository $repo  
24 ) {  
25     $this->repo = $repo;  
26 }  
27
```

Kemudian pada public/index.php:

```
126 $database =  
127     Database::getInstance();  
128  
129  
130 $mahasiswaRepository =  
131     new MahasiswaRepository(  
132         $database  
133     );  
134
```

3.7 Implementasi Base Model

```
1 <?php  
2  
3 namespace App\Core;  
4  
5 use PDO;  
6  
7 0 references | 0 implementations  
8 abstract class BaseModel  
9 {  
10     1 reference  
11     protected PDO $db;  
12  
13     0 references | 0 overrides  
14     public function __construct()  
15     {  
16         $this->db =  
17             Database::getInstance()  
18                 ->getConnection();  
19     }  
20 }
```

3.8 Pengujian Aplikasi

Pengujian dilakukan untuk memastikan fitur aplikasi berjalan sesuai dengan instruksi tugas.

a. Pengujian Login

The image shows two screenshots of a web application. The top screenshot is the login page, titled 'Login SI Akademik'. It features a form with 'Username' (admin) and 'Password' (masked with dots) fields, a 'Login' button, and a footer note 'Username: admin | Password: 12345'. The bottom screenshot is the dashboard, titled 'SI Akademik'. It includes a navigation bar with links to 'Dashboard', 'Mahasiswa', 'Prodi', 'Mata Kuliah', and 'Logout'. A green banner says 'Selamat datang, Admin'. Below this, there's a 'Dashboard' section with a welcome message and a table with two buttons: 'Data Mahasiswa' (Kelola data mahasiswa) and 'Tambah Mahasiswa' (Tambahkan data mahasiswa baru).

Username

admin

Password

.....

Login

Username: admin | Password: 12345

© 2026 SI Akademik - Politeknik Negeri Jember

SI Akademik

Dashboard Mahasiswa Prodi Mata Kuliah Logout

Selamat datang, Admin

Selamat datang, Admin

Dashboard

Selamat datang, Admin!

Selamat datang di Sistem Informasi Akademik.

Data Mahasiswa
Kelola data mahasiswa

Tambah Mahasiswa
Tambahkan data mahasiswa baru

2.9 Pengujian Tampil Data

The image shows a screenshot of the 'Data Mahasiswa' page in the SI Akademik application. It features a search bar with the placeholder 'Cari berdasarkan NIM atau nama...' and a 'Cari' button. Below the search bar is a table with 8 columns: No, NIM, Nama, Email, Program Studi, Angkatan, Status, and Aksi. The table contains 4 rows of student data. At the bottom, there is a message 'Menampilkan 4 dari 4 data mahasiswa.' and a button '← Kembali ke Dashboard'.

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	250004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TK - Teknik Komputer	2025	Aktif	Detail Edit Hapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 4 dari 4 data mahasiswa.

← Kembali ke Dashboard

2.10 Pengujian Tambah Data

localhost/Web%20Server/Acara%2010/si-akademik/public/mahasiswa/create

Tambah Data Mahasiswa

NIM
250004

Nama Mahasiswa
Nila Agustin Prima Aurelia

Email
nilaaurelia97@gmail.com

Program Studi
TI - Teknik Informatika

Angkatan
2046

Status Mahasiswa
Aktif

Kembali Simpan

SI Akademik Dashboard Mahasiswa Prodi Mata Kuliah Logout

Data Mahasiswa + Tambah Mahasiswa

Cari berdasarkan NIM atau nama... Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	250004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 4 dari 4 data mahasiswa.

[← Kembali ke Dashboard](#)

2.11 Pengujian Update Data

SI Akademik Dashboard Mahasiswa Prodi Mata Kuliah Logout

Edit Data Mahasiswa

NIM
260004

Data mahasiswa berhasil diubah.

Data Mahasiswa + Tambah Mahasiswa

Cari berdasarkan NIM atau nama... Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	260004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2026	Aktif	Detail Edit Hapus

2.12 Pengujian Hapus Data

SI Akademik

localhost says
Yakin ingin menghapus data mahasiswa ini?

Yakin ingin menghapus data mahasiswa ini?

OKCancel

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	260004	Nila Agustin Prima Aurelia	nilaaurelia97@gmail.com	TI - Teknik Informatika	2026	Aktif	Detail Edit Hapus
2	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
3	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
4	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

SI Akademik

DashboardMahasiswaProdiMata KuliahLogout

Data mahasiswa berhasil dihapus.

+ Tambah Mahasiswa

Cari berdasarkan NIM atau nama...

Cari

No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	242003	Citra Lestari	citra@email.com	SI - Sistem Informasi	2024	Aktif	Detail Edit Hapus
2	251002	Ani Wijaya	ani@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus
3	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 3 dari 4 data mahasiswa.

2.13 Pengujian Pencarian

SI Akademik

localhost/Web%20Server/Acara%2010/si-akademik/public/mahasiswa?q=budi

DashboardMahasiswaProdiMata KuliahLogout

Data Mahasiswa

+ Tambah Mahasiswa

budi

Cari

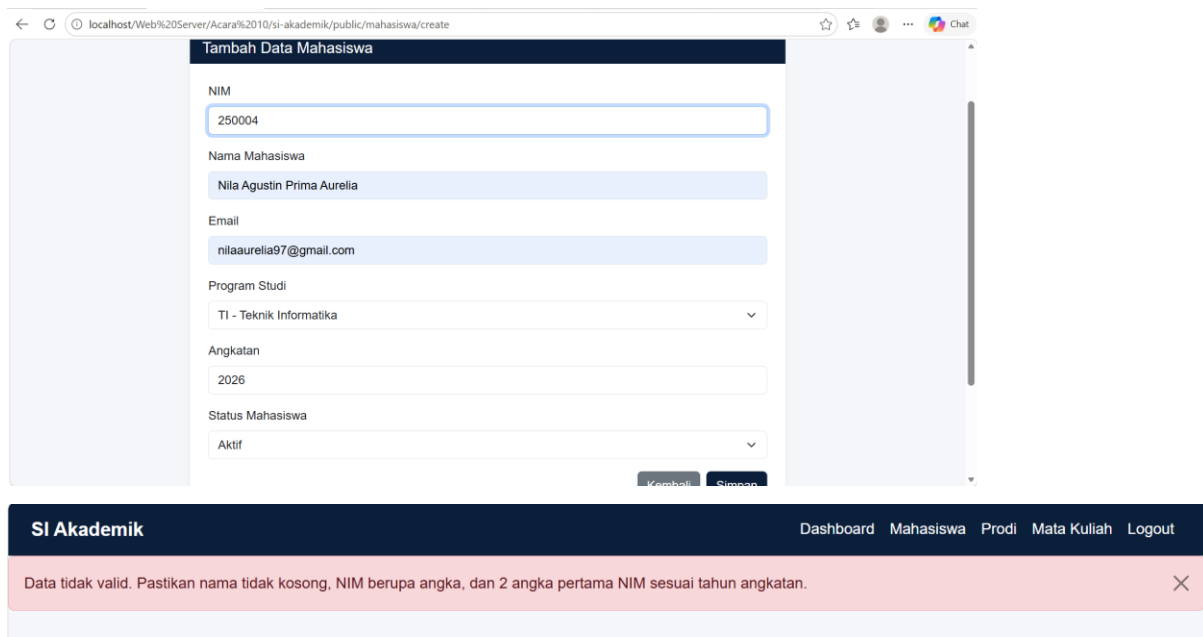
No	NIM	Nama	Email	Program Studi	Angkatan	Status	Aksi
1	251001	Budi Santoso	budi@email.com	TI - Teknik Informatika	2025	Aktif	Detail Edit Hapus

Menampilkan 1 dari 1 data mahasiswa.

Kembali ke Dashboard

© 2026 SI Akademik - Politeknik Negeri Jember

2.14 Pengujian Validasi NIM & Angkatan



The screenshot displays a web browser window with the URL `localhost/Web%20Server/Acara%2010/si-akademik/public/mahasiswa/create`. The page title is "Tambah Data Mahasiswa". The form contains the following fields:

- NIM:
- Nama Mahasiswa:
- Email:
- Program Studi:
- Angkatan:
- Status Mahasiswa:

At the bottom of the form are two buttons: "Kembali" and "Simpan". Below the form, a red error message is displayed: "Data tidak valid. Pastikan nama tidak kosong, NIM berupa angka, dan 2 angka pertama NIM sesuai tahun angkatan." The top navigation bar includes "SI Akademik" and links for "Dashboard", "Mahasiswa", "Prodi", "Mata Kuliah", and "Logout".

Bab 3. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, konsep **Inheritance dan Repository Pattern** berhasil diterapkan pada aplikasi Manajemen Data Mahasiswa.

Konsep Inheritance diterapkan dengan membuat BaseController sebagai parent class yang menyediakan method umum seperti `view()` dan `redirect()`. Controller kemudian mewarisi BaseController, sedangkan MahasiswaController mewarisi Controller. Dengan cara tersebut, method yang memiliki fungsi umum tidak perlu ditulis berulang kali pada setiap Controller.

Repository Pattern diterapkan dengan membuat MahasiswaRepository sebagai class yang bertanggung jawab terhadap seluruh proses akses data mahasiswa ke database. Proses menampilkan, mencari, menambah, mengubah, dan menghapus data dilakukan melalui method Repository. Controller hanya memanggil method yang tersedia pada Repository sehingga query SQL tidak ditulis secara langsung di dalam Controller.

Selain itu, Dependency Injection diterapkan dengan memberikan object MahasiswaRepository kepada MahasiswaController melalui constructor. Hal tersebut membuat hubungan antara Controller, Repository, dan Database menjadi lebih terstruktur.

Berdasarkan hasil pengujian, fitur utama aplikasi seperti **tampil data, tambah data, ubah data, hapus data, pencarian, serta validasi data mahasiswa** dapat dijalankan. Penerapan Inheritance mengurangi duplikasi kode pada Controller, sedangkan Repository Pattern memisahkan logika akses database dari logika aplikasi. Dengan demikian, struktur aplikasi menjadi lebih rapi, mudah dipelihara, dan lebih mudah dikembangkan.

